DERIVATIVE PRICING

MODULE 7 | LESSON 1

STOCHASTIC VOLATILITY: HESTON

Reading Time	50 minutes
Prior Knowledge	Black-Scholes, Local-volatility, Stochastic-volatility
Keywords	Stochastic volatility, Heston model, Skewness, Kurtosis

---

*In Lesson 1 of Module 7, we start dealing with **stochastic volatility models**. Specifically, we will implement the **Monte-Carlo simulation** of the **Heston (1993)** stochastic volatility model.*

As usual, let's start by importing the necessary libraries:

In [10]:
```
import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as ss
```

1. Simulating the Heston Model in Python

As you have already seen in the slides, we can simulate stock price paths under Heston model dynamics with the following two equations: $$ \begin{equation*} S_t = S_{t-1} e^{\left(r - \frac{\nu_t}{2} \right) dt + \sqrt{\nu_t} dZ_1} \end{equation*} $$ $$ \begin{equation*} \nu_t = \nu_{t-1} + \kappa \left(\theta - \nu_{t-1} \right) dt + \sigma \sqrt{\nu_{t-1}}dZ_2 \end{equation*} $$
Importantly, these equations require some parameters, whose functional form is currently unknown to us. At this step, we will just give some values to these parameters to generate sample paths. In the next lesson, we will look at some semi-analytical ways (e.g., Fourier transform methods) to obtain a functional form for these.

We will start by coding the **stochastic volatility function**, ν_t:

Notice how we calculate ν_t as the maximum between the ν_t expression above and 0. This is to ensure that we do not get a negative number here because of the random process. Obviously, it does not make any sense whatsoever to have negative variance!

In [11]:
```
def SDE_vol(v0, kappa, theta, sigma, T, M, I, rand, row, cho_matrix):
    np.random.seed(2)
    dt = T / M  # T = maturity, M = number of time steps
    v = np.zeros((M + 1, I), dtype=float)
    v[0] = v0
```

```
    sdt = np.sqrt(dt)  # Sqrt of dt
    for t in range(1, M + 1):
        ran = np.dot(cho_matrix, rand[:, t])
        v[t] = np.maximum(0, v[t - 1] + kappa * (theta - v[t - 1]) * dt + np.sqr
    return v
```

Next, let's implement the classic **stochastic equation** for the underlying asset price evolution:

In [12]:
```
def Heston_paths(S0, r, v, row, cho_matrix):
    np.random.seed(2)
    S = np.zeros((M+1, I), dtype=float)
    S[0] = S0
    sdt = np.sqrt(dt)
    for t in range(1, M+1, 1):
        ran = np.dot(cho_matrix, rand[:,t])
        S[t] = S[t - 1] * np.exp((r - 0.5 * v[t-1]) * dt + np.sqrt(v[t-1]) * ran

    return S
```

The last function we will define, just for simplifying the tasks, will be a **random number generator** following a standard normal:

In [13]:
```
def random_number_gen(M, I):
    np.random.seed(2)
    rand = np.random.standard_normal((2, M+1, I))
    return rand
```

Now that we have defined the main functions to use in the process, let's implement the Heston model under some parameters (assume these are given for now):

In [14]:
```
v0 = 0.05
kappa_v = 2
sigma_v = 0.3
theta_v = 0.04
rho = -0.9

S0 = 100  # Current underlying asset price
r = 0.05  # Risk-free rate
M0 = 5   # Number of time steps in a year
T = 1  # Number of years
M = int(M0*T) # Total time steps
I = 1  # Nomber of simulations
dt = T/M  # Length of time step
```

Finally, let's work on the random number generator and the covariance matrix (using Cholesky decomposition to account for correlation between dZ_1 and dZ_2):

Finally, let's work on the random number generator and the covariance matrix (using Cholesky decomposition to account for correlation between dZ_1 and dZ_2):

In [15]:
```
# Generating random numbers from standard normal
np.random.seed(2)
rand = random_number_gen(M, I)


# Covariance Matrix
```

```
covariance_matrix = np.zeros((2, 2))
covariance_matrix[0] = [1.0, rho]
covariance_matrix[1] = [rho, 1.0]
cho_matrix = np.linalg.cholesky(covariance_matrix)
```

Now we have all the ingredients to generate the paths for both asset price and its volatility:

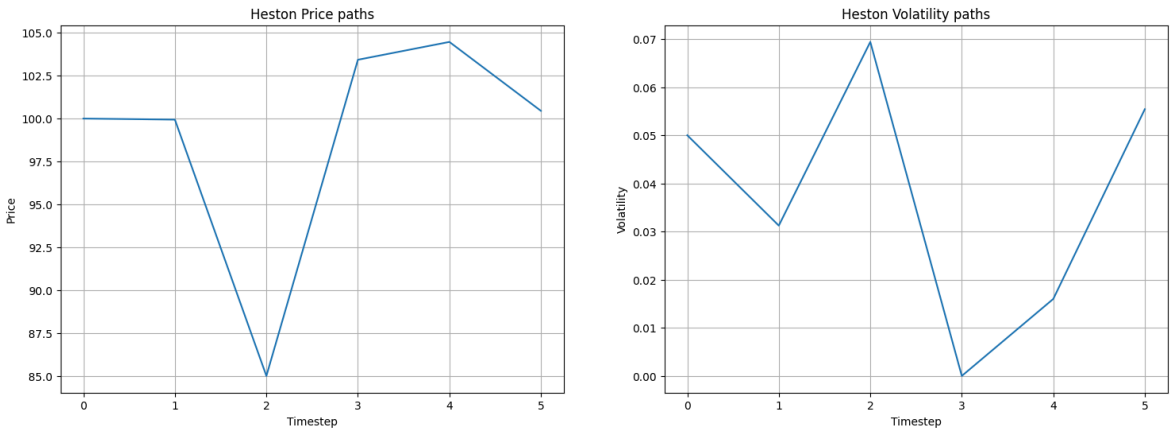
In [16]:
```
# Volatility process paths
V = SDE_vol(v0, kappa_v, theta_v, sigma_v, T, M, I, rand, 1, cho_matrix)

# Underlying price process paths
S = Heston_paths(S0, r, V, 0, cho_matrix)
```

In [17]:
```
S
```

Out[17]:
```
array([[100.        ],
       [ 99.93735136],
       [ 84.99390527],
       [103.42812163],
       [104.46759154],
       [100.45003833]])
```

Let's visualize some of the paths for both the underlying price and the volatility:

In [18]:
```
def plot_paths(n):
    fig = plt.figure(figsize=(18, 6))
    ax1 = fig.add_subplot(121)
    ax2 = fig.add_subplot(122)

    ax1.plot(range(len(S)), S[:, :n])
    ax1.grid()
    ax1.set_title("Heston Price paths")
    ax1.set_ylabel("Price")
    ax1.set_xlabel("Timestep")

    ax2.plot(range(len(V)), V[:, :n])
    ax2.grid()
    ax2.set_title("Heston Volatility paths")
    ax2.set_ylabel("Volatility")
    ax2.set_xlabel("Timestep")


plot_paths(100)
```



2. The Statistical Distribution Produced by Heston

One important feature of the Heston model is that it produces a distribution of returns that has **heavier tails and kurtosis** than a normal distribution, just as we observe in practice.

In the following code snippet, we will check the Heston-produced distribution on two fronts:

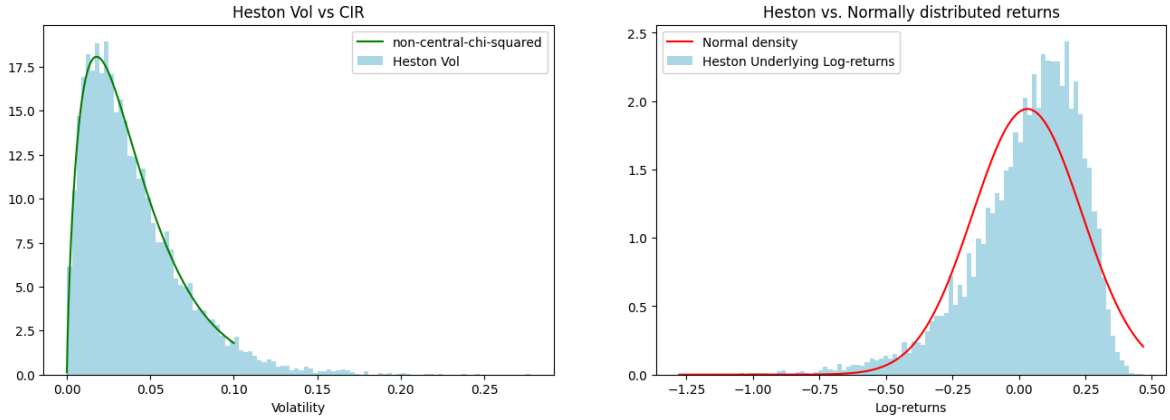
1. Whether **underlying returns** resemble a Normal distribution, as assumed in BS and GBM process.

2. How **volatility** fits a mean-reverting process such as CIR or Vasicek.

In [20]:
```
# Obtaining degrees of freedom (do not worry a lot about this now)
c = 2 * kappa_v / ((1 - np.exp(-kappa_v * T)) * sigma_v**2)
df = 4 * kappa_v * theta_v / sigma_v**2
nc = 2 * c * v0 * np.exp(-kappa_v * T)


# Calculating returns and lengths of axis
log_R = np.log(S[-1, :] / S0)
x = np.linspace(log_R.min(), log_R.max(), 500)
y = np.linspace(0.00001, 0.1, 500)


# Plotting the different distributions
fig = plt.figure(figsize=(16, 5))
ax1 = fig.add_subplot(121)
ax2 = fig.add_subplot(122)

# Heston stochastic vol follows a CIR/Vasicek process
ax1.plot(
    y,
    ss.ncx2.pdf(y, df, nc, scale=1 / (2 * c)),
    color="green",
    label="non-central-chi-squared",
)
ax1.hist(V[-1, :], density=True, bins=100, facecolor="LightBlue", label="Heston
ax1.legend()
ax1.set_title("Heston Vol vs CIR")
ax1.set_xlabel("Volatility")

# Heston underlying returns do not follow a normal distribution
ax2.plot(
    x,
    ss.norm.pdf(x, log_R.mean(), log_R.std(ddof=0)),
    color="r",
    label="Normal density",
)
ax2.hist(
    log_R,
    density=True,
    bins=100,
```

```
    facecolor="LightBlue",
    label="Heston Underlying Log-returns",
)
ax2.legend()
ax2.set_title("Heston vs. Normally distributed returns")
ax2.set_xlabel("Log-returns")
```

Out[20]: Text(0.5, 0, 'Log-returns')



In the previous graphs, you can observe the fit (and no-fit) of the different distributions produced to known distributions assumed in other models.

3. Conclusion

Well done! Now you know a lot more about how to implement the Heston model in Python. Arguably, one thing we have not yet covered explicitly is pricing in the Heston framework. This is exactly what we will do in the next lesson.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.